

Deep Learning for NLP

Student name: *Angelos Dorotheos Chatzopoulos*
cs: 7115112400027

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Fall Semester 2025*

Contents

1	Abstract	3
2	Data Analysis and Preprocessing	3
2.1	Data initial analysis	3
2.1.1	Dimensions of datasets and checking for NULL or duplicated rows	3
2.1.2	Distribution of sentiment labels	3
2.1.3	Distribution of sentiment labels	4
2.1.4	Tweets length analysis before preprocessing	5
2.1.5	Finding top 10 unigrams, bigrams & trigrams before preprocessing	5
2.1.6	Counting emails and user mentions in tweets	6
2.2	Tweets preprocessing & further analysis	6
2.2.1	Tweets length after preprocessing with stop words filtering enabled	8
2.2.2	Distribution of sentiment labels after applying preprocessing . .	9
2.2.3	Finding top 10 unigrams, bigrams & trigrams after applying pre- processing	10
3	Word2Vec Embeddings - Feature Extraction	12
3.1	Dataset configurations	12
3.2	Word2Vec embeddings generation	12
3.3	Neural network training	12
3.4	Performance evaluation metrics	13
4	Feed Forward Neural Network Architecture Design and Experiments	13
4.1	Initial network architecture	13
4.2	Preprocessing comparison: stopwords filtered vs preserved	14
4.3	Experimenting with dropout in our initial architecture	15
4.4	Hidden layers experimentation	16
4.5	Optimizer Tuning	27
4.6	Batch Normalization	32
5	Optuna and Final Results	34
5.1	Running an Optuna study	34
5.2	Comparison with the first project & future work	35

6 Bibliography

37

1. Abstract

This project develops a sentiment analysis classifier using deep neural networks and Word2Vec embeddings for feature extraction on an English-language Twitter dataset. The datasets consist of three features: ID, text, and label, where the label shows the positive sentiment (1) or negative sentiment (0) for each tweet. Similar to the first project, an exploratory data analysis (EDA) is performed on a train set and a validation set to understand the distribution of labels and characteristics of datasets' tweets. Next, based on the data analysis, text preprocessing is applied to remove any unnecessary parts of the text. The tweet texts are then converted into word embeddings using the pre-trained GoogleNews Word2Vec corpus, and their mean values are calculated, to represent each tweet by a 300-dimensional word embedding. Moreover, a PyTorch neural network is implemented, and experiments are conducted to tune the hyperparameters of the model's architecture. Finally, by using multiple performance metrics such as accuracy, precision, recall, F1-score, as well as test and validation set loss, a final model is selected.

2. Data Analysis and Preprocessing

2.1. Data initial analysis

Initially, an in-depth analysis is performed to find the key characteristics of the training and validation datasets [9]. **Since a different preprocessing approach was used from the previous project, the exploratory data analysis steps are redone (although it was not necessary).** This exploratory data analysis (EDA) phase plays a significant role in understanding the structure of our datasets. It allowed us to understand the distribution of sentiment labels, the tweet lengths, the existence of usernames and email addresses within the texts, as well as identifying the most frequently occurring tokens. Visual tools such as histogram plots and word clouds were utilized to guide us through the preprocessing and modeling stages.

2.1.1. Dimensions of datasets and checking for NULL or duplicated rows. In this step, we examine the dimensions of the given datasets. Table 1 shows the dimensions of train validation and test datasets. Training and validation datasets contain three columns (ID, Text, Label) while test dataset contains only 2 columns (ID, Text).

Dataset	Dimensions
Train	(148388, 3)
Validation	(42396, 3)
Test	(21199, 2)

Table 1: Dimensions of Datasets

Furthermore, null values and duplicate rows are examined. Observing Table 2, we notice that there are no missing or duplicate values in both training and validation datasets.

2.1.2. Distribution of sentiment labels. In this step, the distribution of sentiment labels are examined to check for any dataset imbalance. Figure 1 shows that both train-

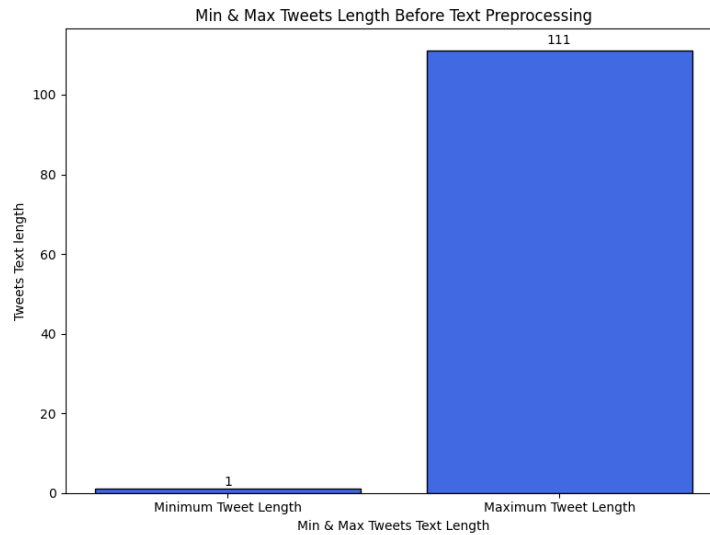


Figure 5: Minimum and maximum length of tweets in training dataset before applying text preprocessing.

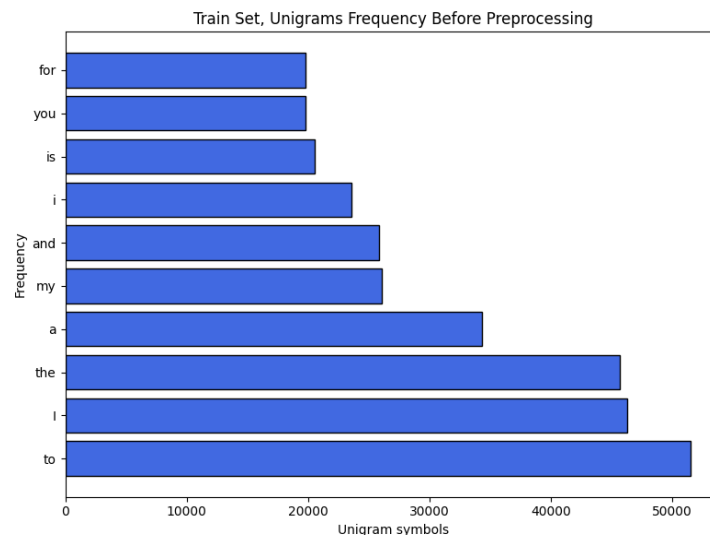


Figure 6: Top 10 unigrams in training set before applying text preprocessing.

2.1.6. Counting emails and user mentions in tweets. Figure 8 shows that a significant number of user mentions are consisted in tweets while the frequency of emails is low. In this project, during preprocessing, these tokens will be completely removed from the tweets compared to the first project where we anonymized them for feature extraction using TF-IDF and for privacy reasons.

2.2. Tweets preprocessing & further analysis

Text preprocessing is a fundamental technique in natural language processing (NLP) that aims to clean the text by running a series of transformations such as lowercase, eliminate and convert tokens. These steps reduce noise and help the model focus on the most important tokens and improve the efficiency of the system improving the model's accuracy and generalization overall. Compared to the previous project, we

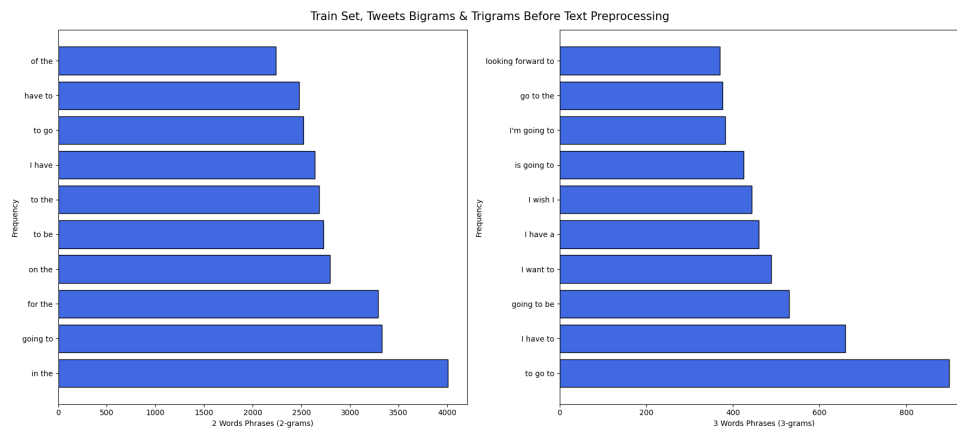


Figure 7: Top 10 bigrams and trigrams in training set before applying text preprocessing.

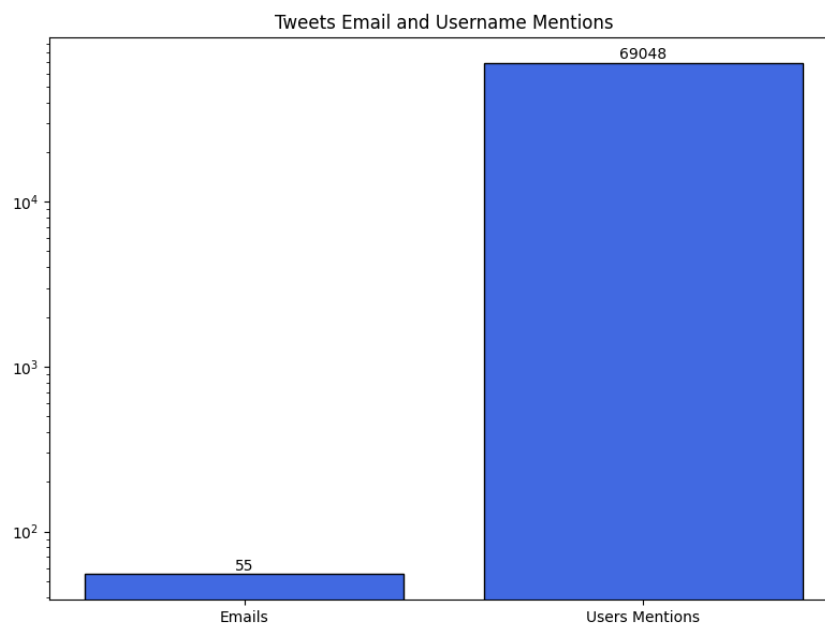


Figure 8: Counting email and user mentions in tweets before preprocessing on training dataset.

improved the preprocessing task by also filtering out username mentions and email addresses, as well as expanding common internet slang (e.g., converting *lol* to *laughing out loud*). This was done for the pre-trained GoogleNews corpus to find and convert as many tokens as it can before calculating the final mean embedding vector for each tweet. Tweets preprocessing experiments on our neural network models are performed in two phases:

- Preprocessing with stop words filtering enabled (where we perform further EDA).
- Preprocessing with stop words preservation enabled (where we notice later better accuracy scores and generalization by preserving them).

A comprehensive pre-process approach is applied on tweets [6, 2, 5, 4, 8]. The preprocessing steps applied on all the datasets (train, validation, test) are the following:

- **Removing Non-ASCII Characters:** Non-ASCII characters are removed from tweets.
- **Lowercasing:** All characters in the text are converted to lowercase.
- **Expanding Contractions:** Shortened words are expanded to their full forms without apostrophes (e.g., *didn't* becomes *did not*).
- **Removing Punctuation:** All punctuation, except for '@' (used for user mentions and email addresses), is removed from the text.
- **Removing Mentions:** User mentions (e.g., @john) are now also removed compared to the previous project where these tokens were anonymized with a '@user' token. This is done because GoogleNews embeddings do not include any *tweet reference* token embedding to be mapped to.
- **Removing Email Addresses:** Email addresses are now removed compared to the previous project where these tokens were anonymized with a 'XXX@email.com' token. This is done because GoogleNews embeddings do not include any email token embedding to be mapped to.
- **Removing URLs:** Any URLs present in the text are removed.
- **Removing Digits:** All numeric characters are removed from the text.
- **Filtering Stop Words:** If stop word filtering is enabled, common stop words are removed from the text.
- **Lemmatization:** Words are converted to their root form (e.g., *studying* becomes *study*).
- **Filtering/expanding slang words:** Common slang terms are detected and replaced [8] with their full meaning to improve consistency in the data (e.g., *lol* to *laughing out loud*).
- **Removing Empty Rows:** Any row with zero text length after applying the above steps is removed (used only to the training and validation sets).

2.2.1. Tweets length after preprocessing with stop words filtering enabled. Due to the removal of stop words, the length of tweets has decreased. Observing Figure 9, we notice that most tweets contain between 5 - 10 words with a mean value of 7.02. Furthermore, Figure 10 shows the minimum and maximum length of tweets in the training dataset after applying preprocessing. We notice that the maximum length of tweets has decreased from 111 to 30. Compared to the previous assignment, the maximum tweets length has been increased (from 28 to 30) due to the expansion of slang words that later will help us achieve high validation set accuracy by mapping as many tokens from tweets to the corresponding embedding vectors from GoogleNews.



Figure 12: Word cloud representations of training set tweets after preprocessing: Positive sentiment (left) and Negative sentiment (right).

2.2.2. Distribution of sentiment labels after applying preprocessing. We show the most common words in the training dataset using wordclouds after running preprocessing with stop words filtering enabled. Figure 11, shows the overall distribution of words from the entire training dataset. The second figure, Figure 12, presents two separate word clouds: one for tweets with positive sentiment and another for tweets with negative sentiment.

After examining the word clouds with preprocessing enabled, we notice that the most common words are *today*, *love* and *laughing* overall. In addition, within the positive data, pleasant words such as *love*, *thank* and *laughing* remain high in frequency. Finally, tweets with negative sentiment analysis highlight words such as *today*, *got* and *work*.

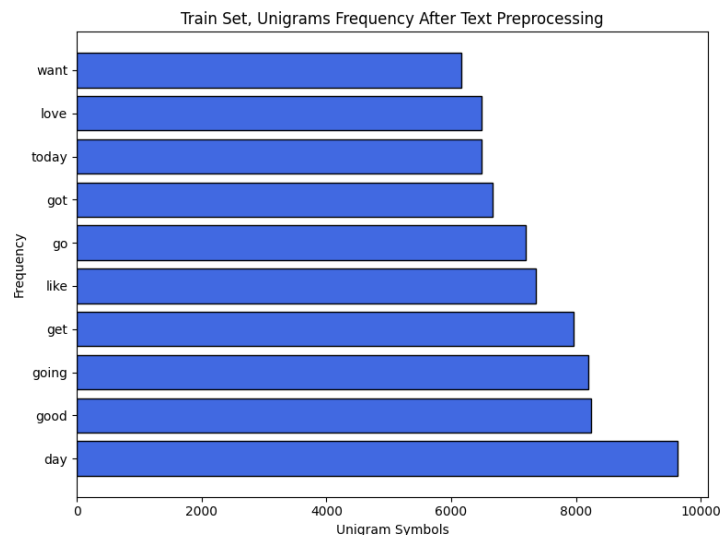


Figure 13: Top 10 unigrams in training set after applying tweets preprocessing.

2.2.3. Finding top 10 unigrams, bigrams & trigrams after applying preprocessing. Figures 13, 14, 15, and 16 show a much clearer and more informative representation of the most frequent words compared to the non-preprocessed plots. This improvement is due to the removal of common stop words, which often dominate the distribution without adding important semantic value, though in our case we will later see that, similarly to the first project, by preserving stop words into the preprocessed dataset it provides better accuracy to our models.

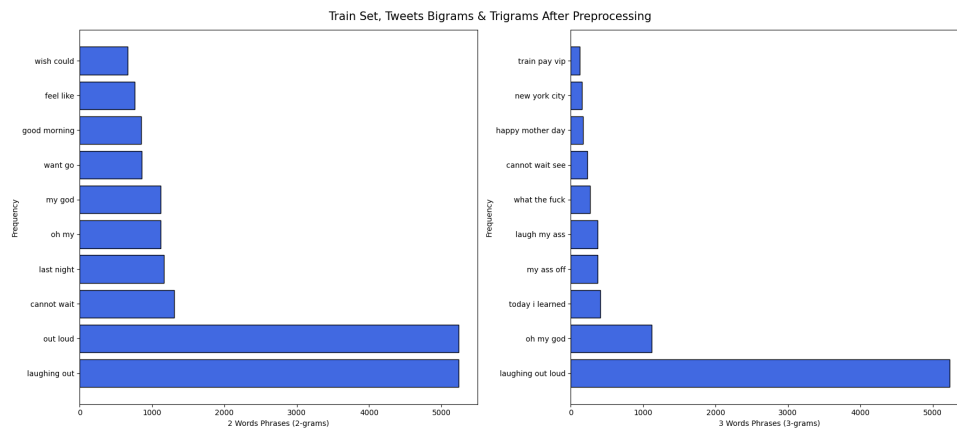


Figure 14: Top 10 bigrams and trigrams in training set after applying tweets preprocessing.

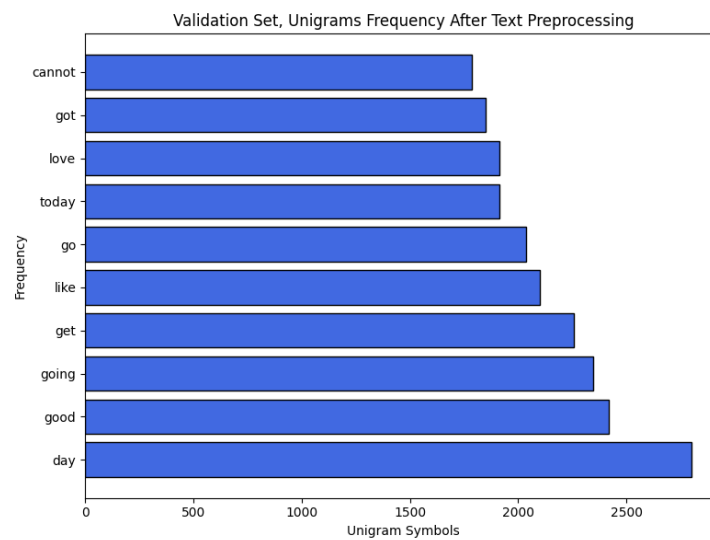


Figure 15: Top 10 unigrams in validation set after applying tweets preprocessing.

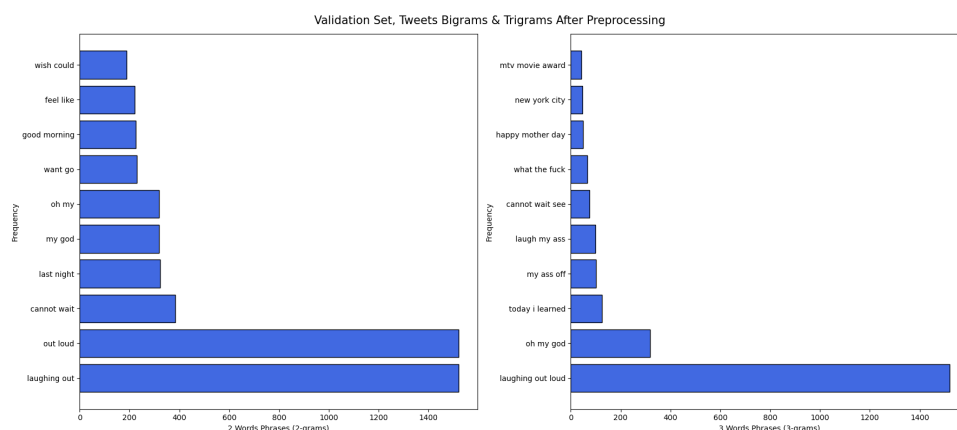


Figure 16: Top 10 bigrams and trigrams in validation set after applying tweets preprocessing.

3. Word2Vec Embeddings - Feature Extraction

Word2Vec is a feature extraction technique that transforms text (tweets) into vector representations of words based on the surrounding words. Before proceeding with the neural network architecture design and hyperparameter tuning, multiple experiments were conducted on Word2Vec configurations while using a basic Feed Forward Neural Network (described thoroughly later). We first experimented with training Word2Vec ourselves, as well as using Word2Vec by GloVe [16] with 100, 200, and 300 dimension embeddings. However, the 300-dimensional embeddings from GoogleNews [7] provided the best results overall. As a result, the experimentation for this project was done using the GoogleNews 300 dimensions Word2Vec embeddings. The neural network model at the start is a basic model, and changes to the model's architecture and hyperparameters are done to gradually improve the performance scores.

3.1. Dataset configurations

The GoogleNews corpus experiment was conducted in two pre-processed dataset configurations:

- A pre-processed dataset with filtered stop words.
- A pre-processed dataset with preserved stop words.

Therefore, the following acronyms are used on the configuration labels for the pre-processed datasets:

- **PRFS**: Preprocessing & Filtering Stopwords
- **PRPS**: Preprocessing & Preserving Stopwords

As we will show on the next section, we noticed that the configuration with preserved stop words provided better accuracy results overall on our validation set, thus we continue our experimentation using this pre-processing configuration.

3.2. Word2Vec embeddings generation

For each tweet, the Word2Vec embeddings are generated by calculating the average from the embeddings of all words in the tweet. This mean value is taken as the final embedding for each tweet text. If a word/token is not included in GoogleNews corpus then it is not used in the tweets embedding creation process. If, for example, the tokens that are not in the GoogleNews corpus are set to a specific vector, this could negatively affect the mean value of the final embedding for each tweet, leading to worse model performance.

3.3. Neural network training

The embeddings (300 dimension mean value embeddings) are then used as input to our feed forward neural network, which is trained using the train set, while the validation set is used for performance evaluation. Learning curves are calculated based on the corresponding train set and validation set based on the number of epochs.

3.4. Performance evaluation metrics

The performance evaluation metrics used on the subsequent feed forward models on each epoch are the following:

- **Train Loss:** Measures the loss on the training dataset.
- **Validation Loss:** Measures the loss on the validation dataset.
- **Train Accuracy:** Measures the proportion of correctly classified instances in the training set.
- **Validation Accuracy:** Measures the proportion of correctly classified instances in the validation set.
- **Precision:** Measures the accuracy of positive predictions.
- **Recall:** Measures the proportion of actual positives that were predicted correctly as positives.
- **F1 Score:** Balances precision and recall and is mainly used for imbalanced datasets detection.

The above performance evaluation scores are evaluated at each epoch to guide us during the training & validation phase of the neural network.

4. Feed Forward Neural Network Architecture Design and Experiments

This section shows the step-by-step experimentation process using multiple Feed Forward Neural Network configurations. The experiments began with an initial architecture and progressed by testing different numbers of hidden layers and neurons and also introducing optimization methods such as dropout, early stopping, batch normalization, optimizer tuning, and a final automated hyperparameter search with the *Optuna* package [10].

4.1. Initial network architecture

The first experiment was done using an initial feed forward neural network with the GoogleNews Word2Vec embeddings. The Word2Vec embeddings used were the 300-dimensional vectors from the GoogleNews corpus. As previously mentioned, these embeddings were averaged per tweet to generate a final 300-dimensional vector that was passed to the neural network model.

The architecture of the initial neural network is the following:

- **Input Layer:** 300 neurons (Word2Vec embedding size using GoogleNews corpus)
- **Hidden Layer 1:** 128 neurons with ReLU activation
- **Hidden Layer 2:** 64 neurons with ReLU activation
- **Output Layer:** 1 neuron (binary classification)

- **Optimizer:** Adam
- **Loss Function:** BCEWithLogitsLoss [12] (Sigmoid & Binary cross entropy loss)
- **Epochs:** 50
- **Batch Size:** 128

4.2. Preprocessing comparison: stopwords filtered vs preserved

The first set of experiments was done to compare the effect of two preprocessing configurations (PRFS vs PRPS). This comparison allowed us to understand whether preserving common stop word tokens contributes positively to sentiment classification in our architecture.

Figures 17 and 18 show that PRPS has in general higher validation set accuracy throughout the epochs. This might be because some stop words carry important sentiment in short texts such as tweet text. However, both models show signs of overfitting, as the training loss continues to decrease while the validation loss increases over the epochs. In addition, we notice a smaller gap between validation accuracy and training accuracy on PRPS datasets compared to PRFS datasets. Finally, we notice that all the remaining validation set scores (precision, recall, F1) increase with the number of epochs, and they are higher compared to the PRFS datasets. **Therefore, similar to the first project, we chose the PRPS datasets to be used for the following experiments, though overfitting needs to be addressed.**

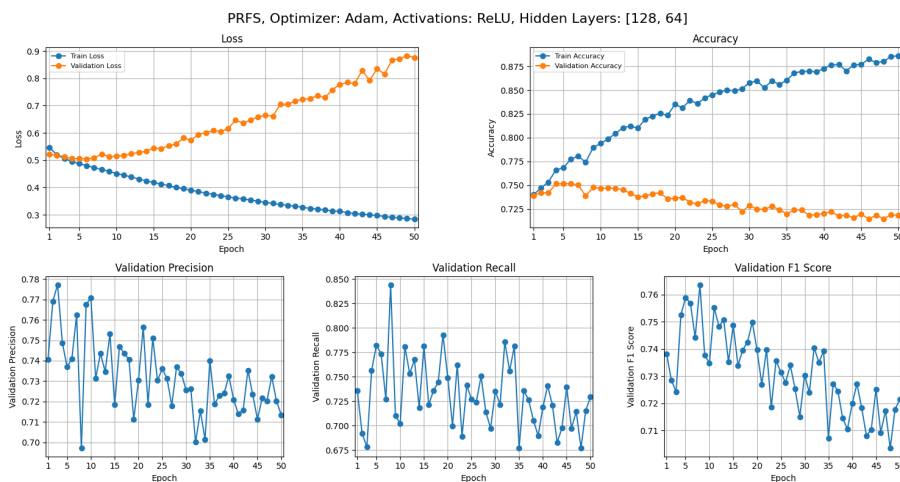


Figure 17: Preprocessing with filtered stop words (PRFS), using the initial neural network model for 50 epochs.

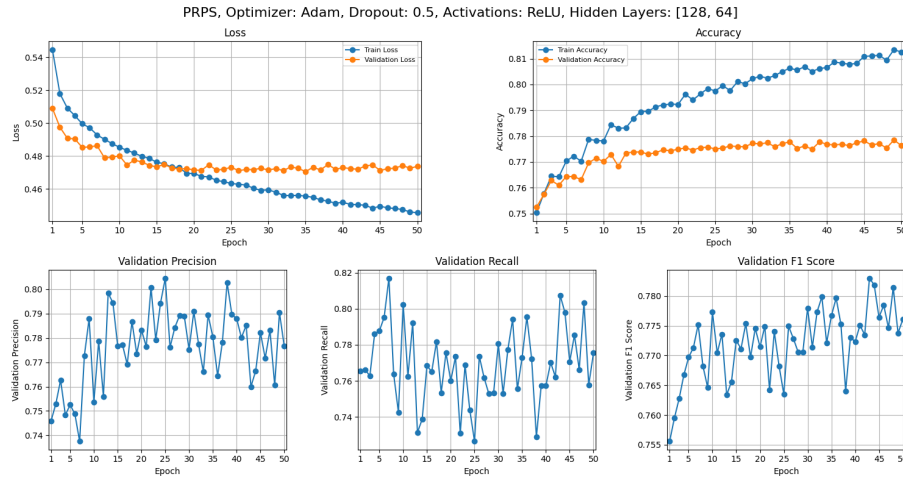


Figure 18: Preprocessing with preserved stop words (PRPS), using the initial neural network model for 50 epochs.

4.3. Experimenting with dropout in our initial architecture

Dropout [13] is a regularization method used to reduce overfitting in neural networks by randomly disabling a fraction of neurons in each training iteration, to prevent the neural network becoming too dependent on certain features. This technique helps the model to become more generalized and perform better on new data examples.

After finding the best preprocessing configuration (PRPS), dropout layers were added to the hidden layers of our initial architecture (Section 4.1) to reduce the overfitting issue. We initially experimented with dropout ratio set to 0.5. Later, we experiment with Optuna to find a more optimal ratio for this hyperparameter on our improved model.

Figure 19 shows that using dropout ratio reduced the overfitting issue, as the distance between training and validation loss became smaller and the validation set accuracy remained more stable throughout the 50 epochs.

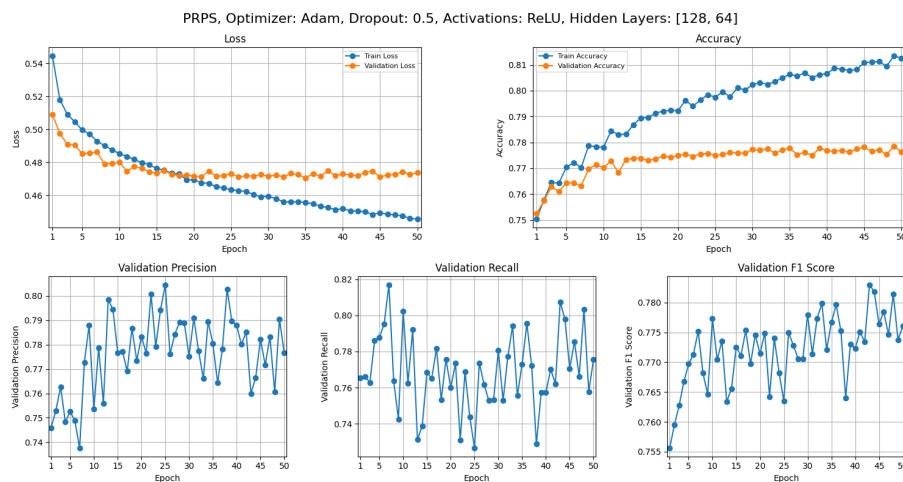


Figure 19: PRPS, with 0.5 dropout ratio, using the initial neural network model for 50 epochs.

4.4. Hidden layers experimentation

After adding dropout layers on each hidden layer to address overfitting, we continued with architectural experiments by modifying the structure of the hidden layers (without modifying any other hyperparameter on the initial model 4.1). In particular, we experimented on different numbers of hidden layers and various neuron sizes in each layer to analyze the performance impact on the models. The following table 3 shows the different hidden layers configurations experimented with.

# Hidden Layers	Neuron Sizes
2	[128, 64]
2	[256, 128]
3	[256, 128, 64]
3	[256, 256, 128]
3	[512, 256, 128]
4	[128, 128, 64, 32]
4	[256, 128, 64, 32]
4	[256, 256, 128, 64]
4	[512, 256, 128, 64]

Table 3: Tested hidden layer configurations

In addition, we introduced early stopping methodology [3] to prevent overfitting and reduce unnecessary training time. Early stopping checks the validation loss during epochs training and stops the training process if the validation loss does not improve for 3 consecutive epochs. **The model with the lowest validation loss found during training is then stored as the best performing model.** The goal was to balance model complexity with generalization performance on the validation set. Using the best performing model, we also display validation set performance metrics, including accuracy, precision, recall, and F1-score. The following figures show the learning curve results on the training and validation sets, as well as the best performance scores (per epoch) on the validation set (Figures 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37).

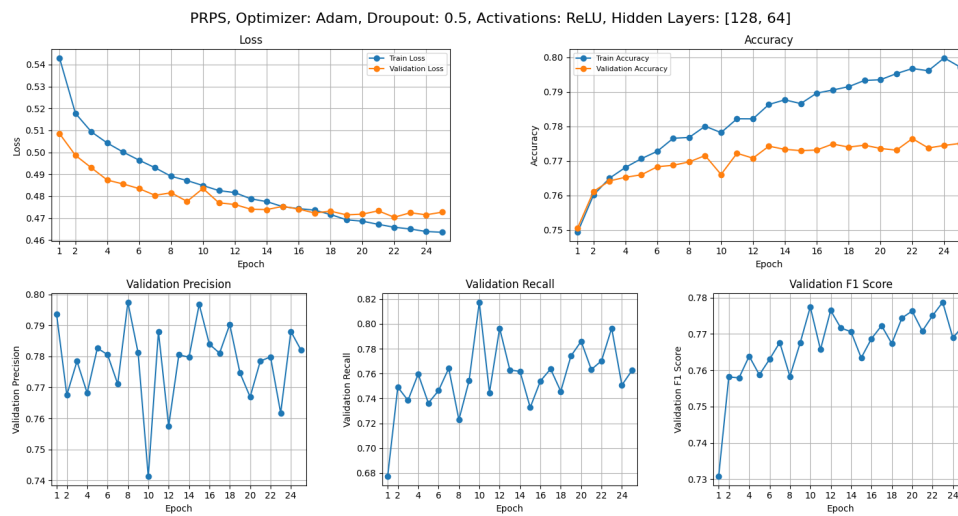


Figure 20: PRPS, 2 hidden layers, [128, 64]

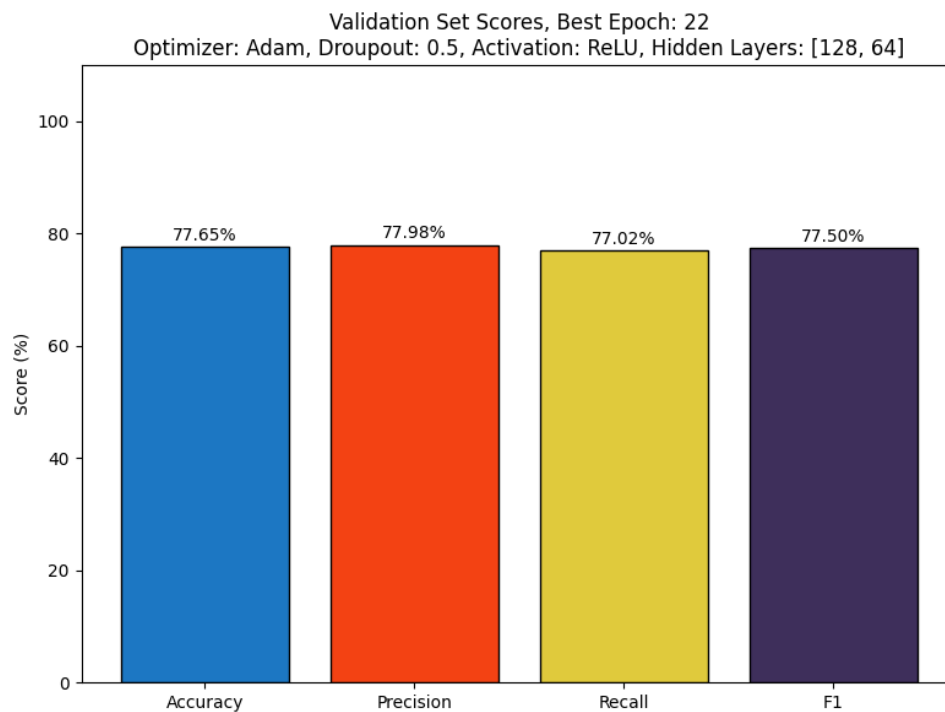


Figure 21: PRPS, best validation scores, 2 hidden layers, [128, 64]

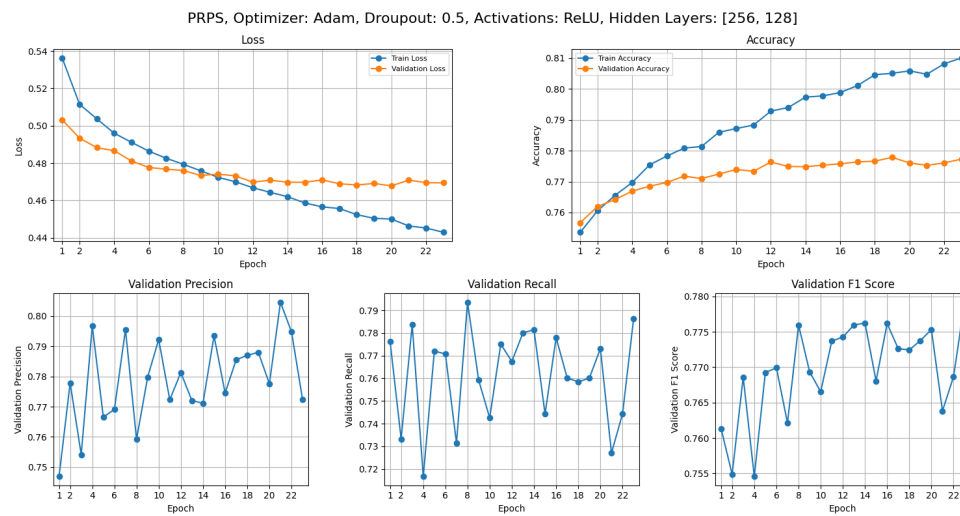


Figure 22: PRPS, 2 hidden layers, [256, 128]

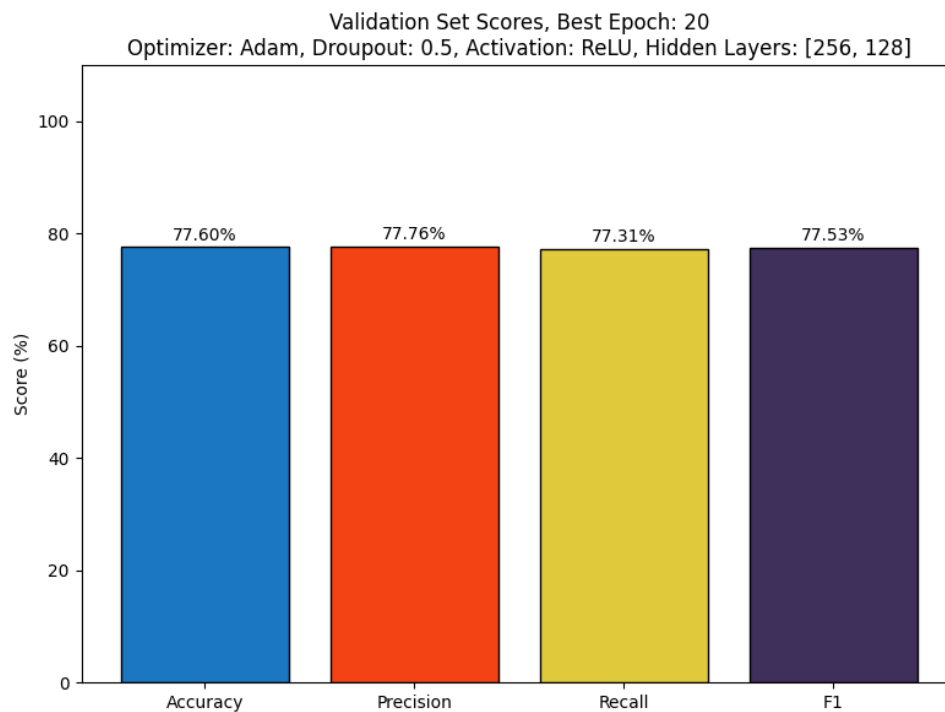


Figure 23: PRPS, best validation scores, 2 hidden layers, [256, 128]

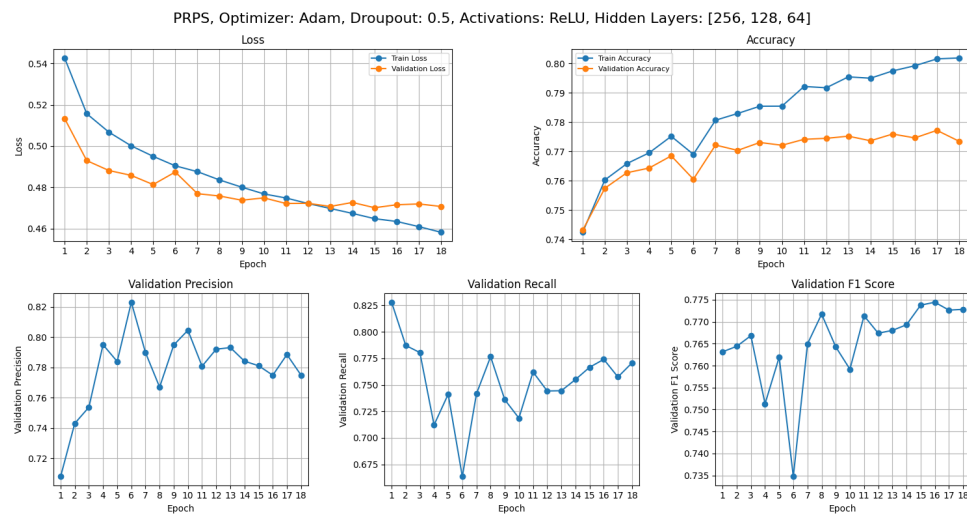


Figure 24: PRPS, 3 hidden layers, [256, 128, 64]

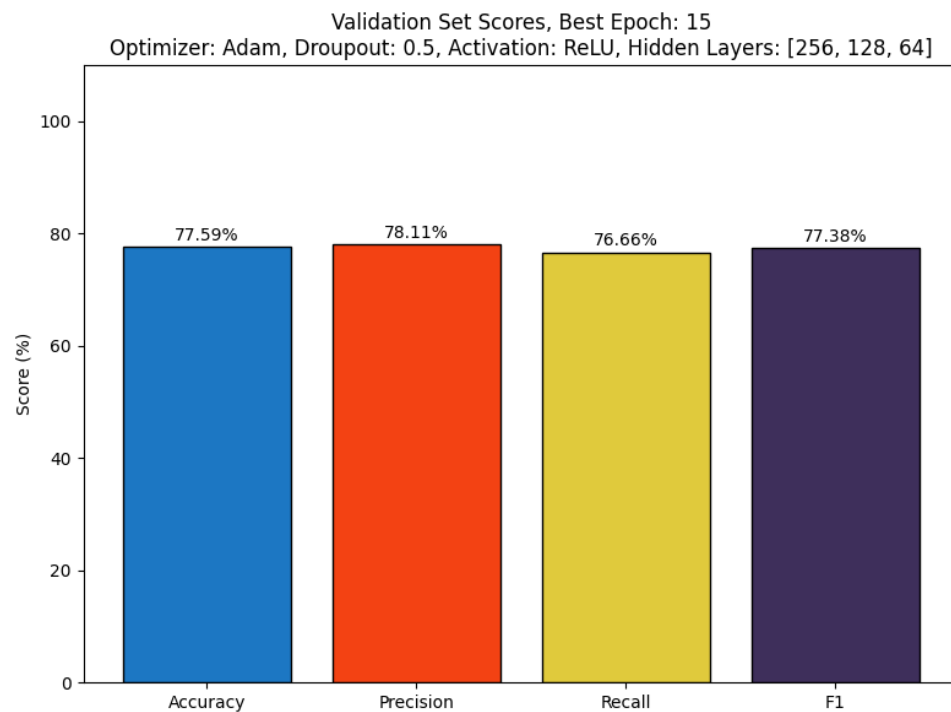


Figure 25: PRPS, best validation scores, 3 hidden layers, [256, 128, 64]

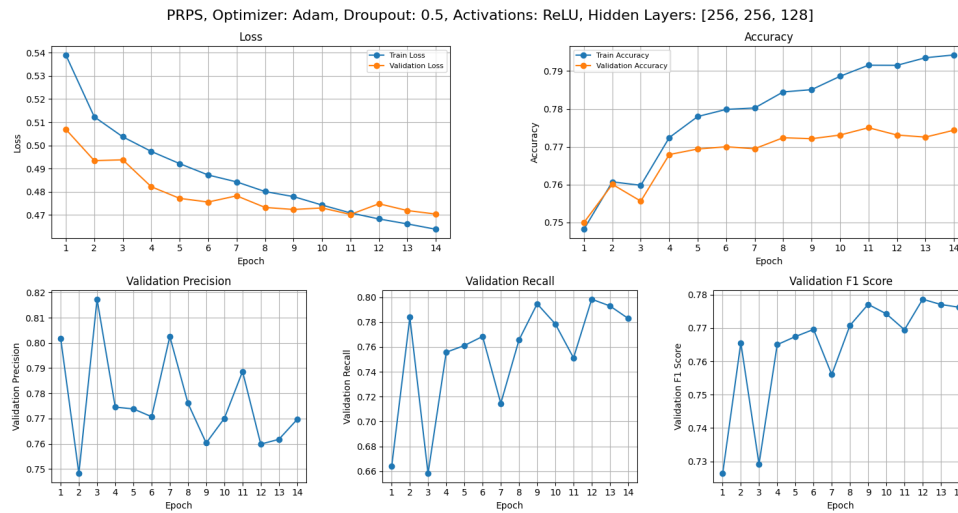


Figure 26: PRPS, 3 hidden layers, [256, 256, 64]

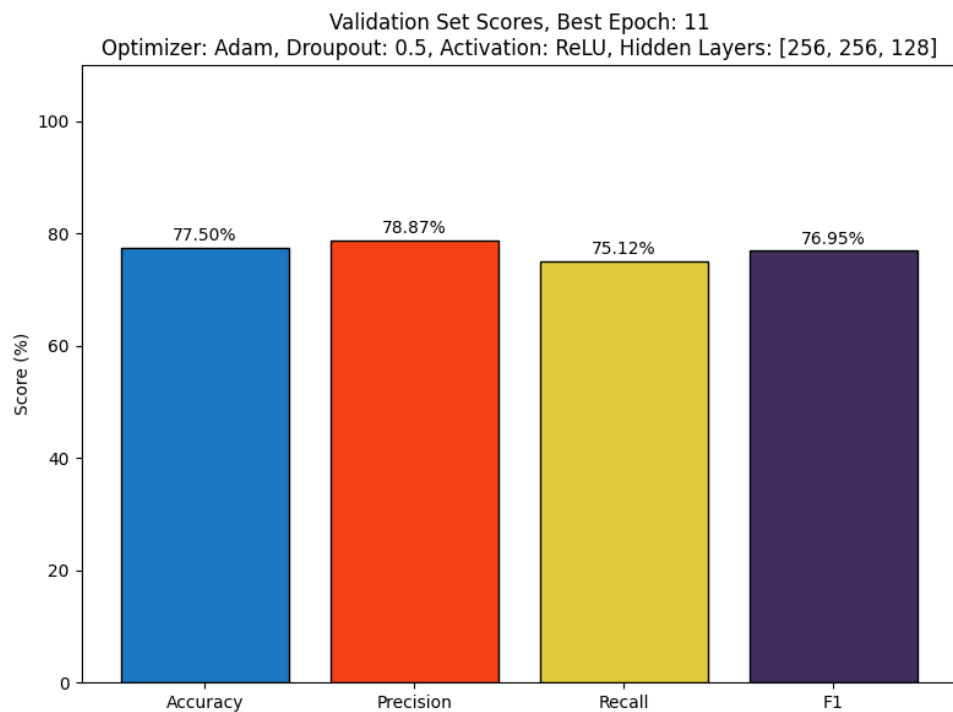


Figure 27: PRPS, best validation scores, 3 hidden layers, [256, 256, 64]

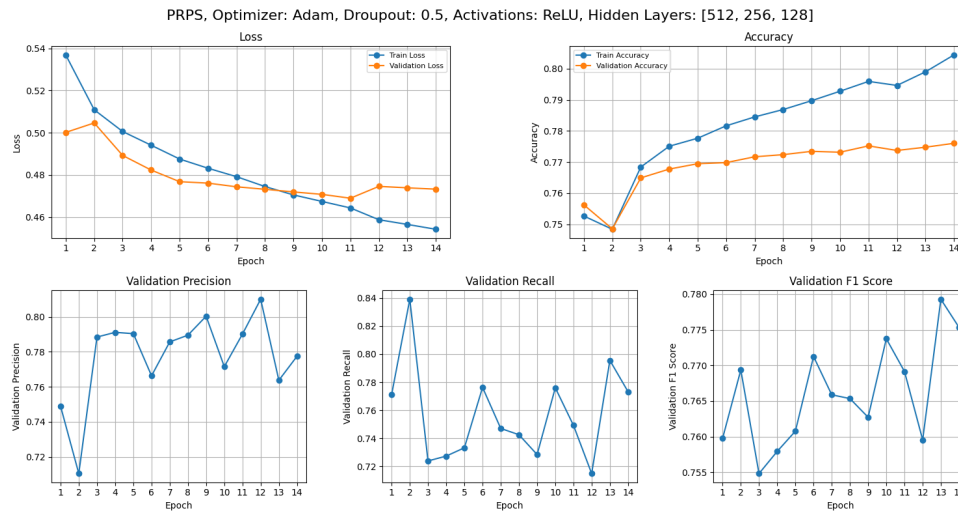


Figure 28: PRPS, 3 hidden layers, [512, 256, 64]

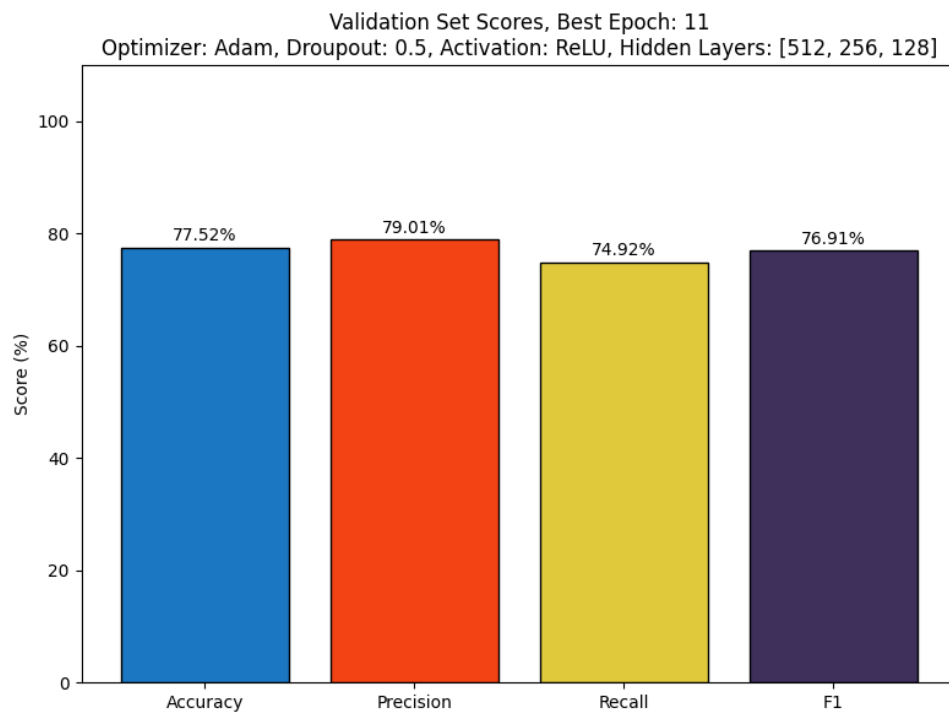


Figure 29: PRPS, best validation scores, 3 hidden layers, [512, 256, 64]

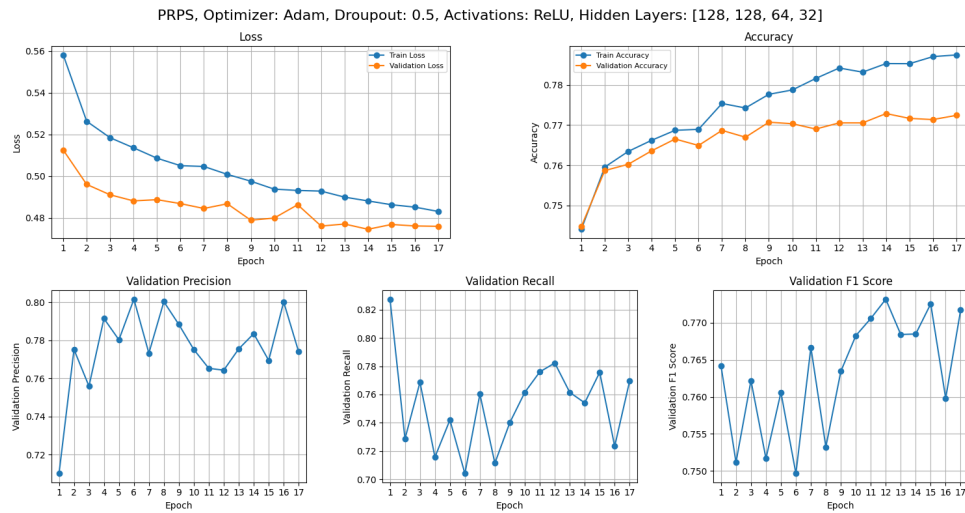


Figure 30: PRPS, 4 hidden layers, [128, 128, 64, 32]

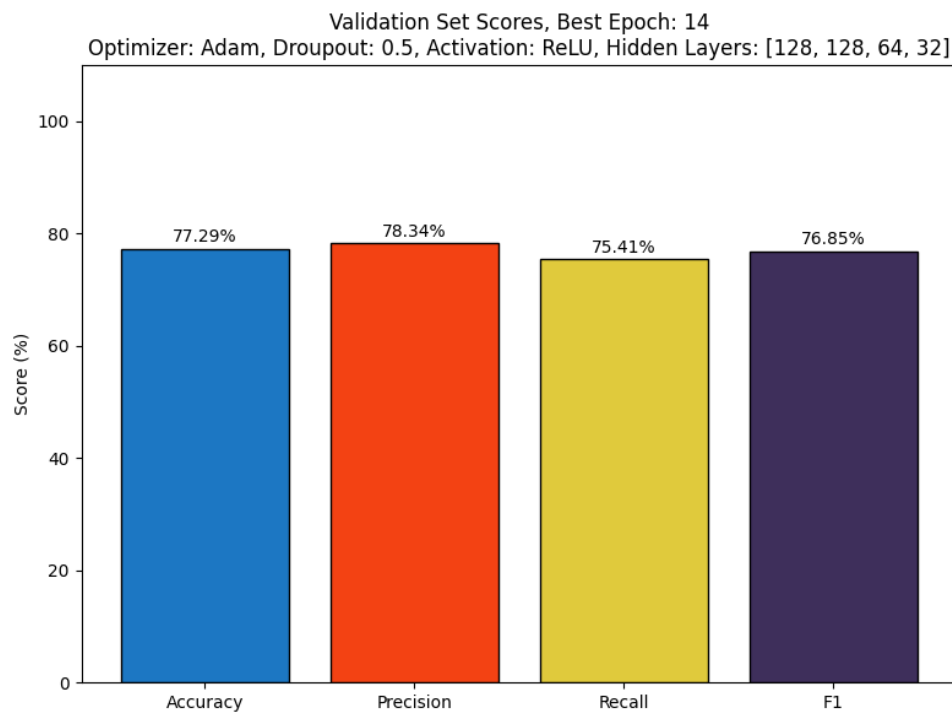


Figure 31: PRPS, best validation scores, 4 hidden layers, [128, 128, 64, 32]

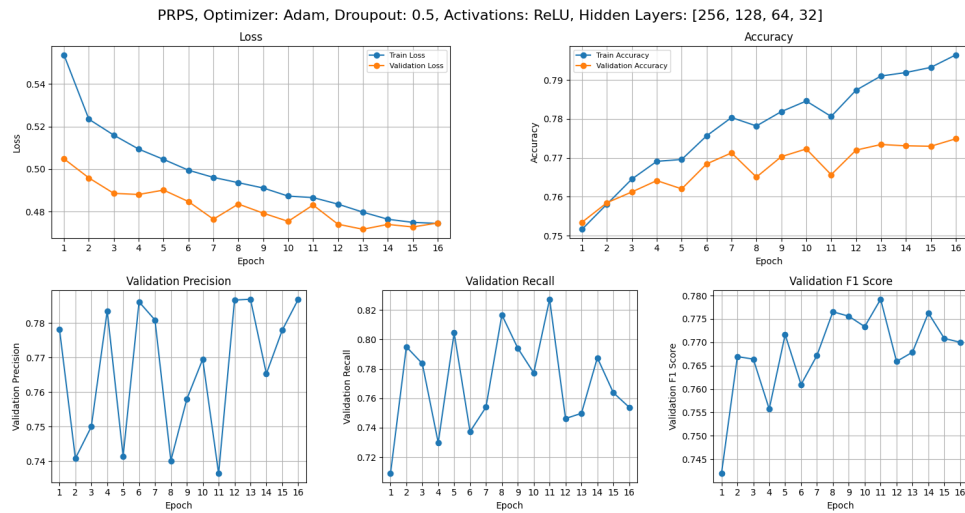


Figure 32: PRPS, 4 hidden layers, [256, 128, 64, 32]

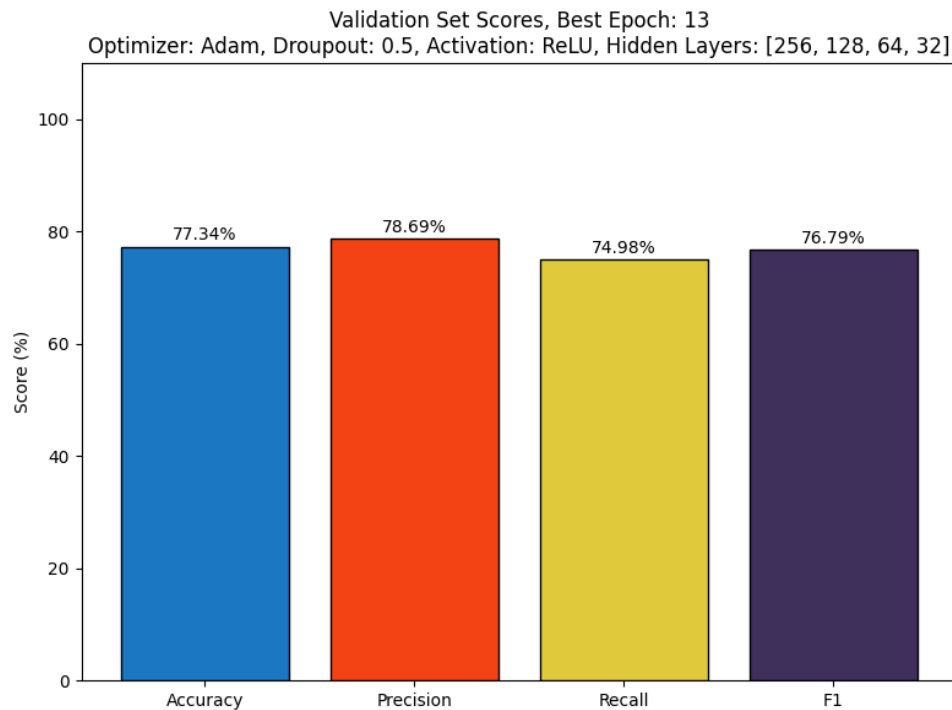


Figure 33: PRPS, best validation scores, 4 hidden layers, [256, 128, 64, 32]

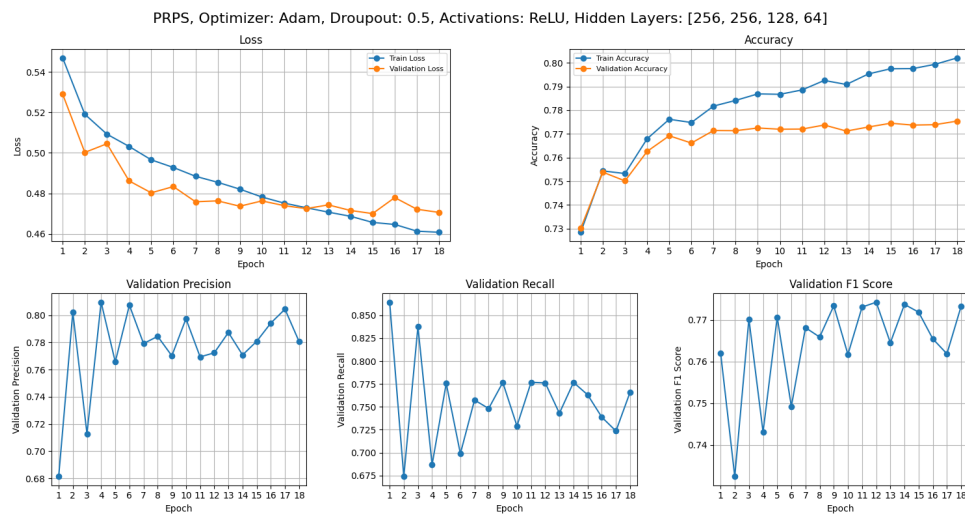


Figure 34: PRPS, 4 hidden layers, [256, 256, 128, 64]

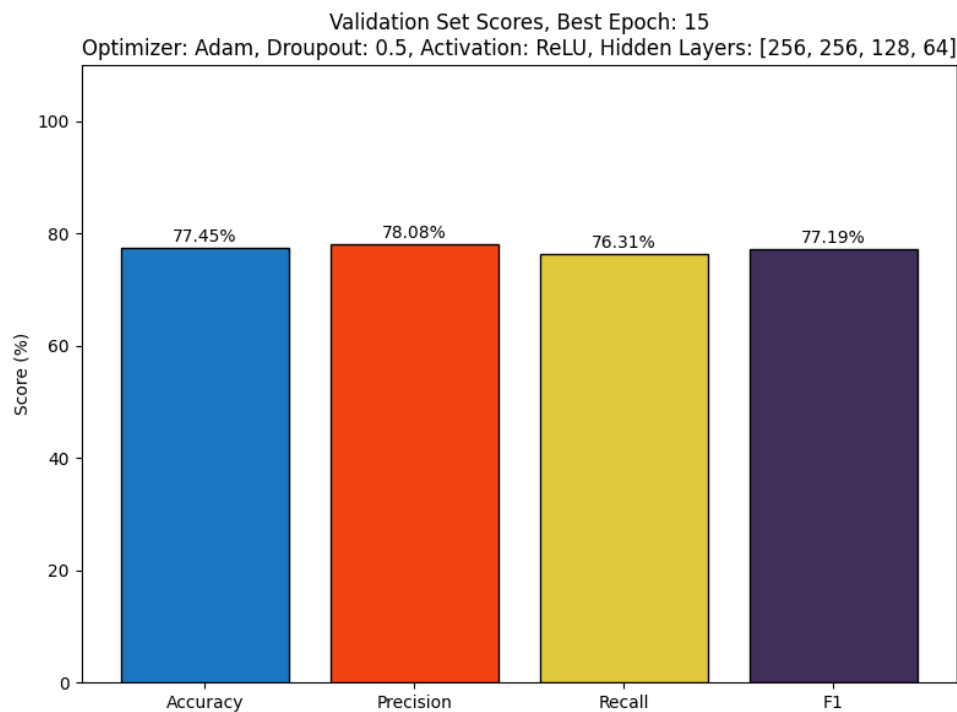


Figure 35: PRPS, best validation scores, 4 hidden layers, [256, 256, 128, 64]

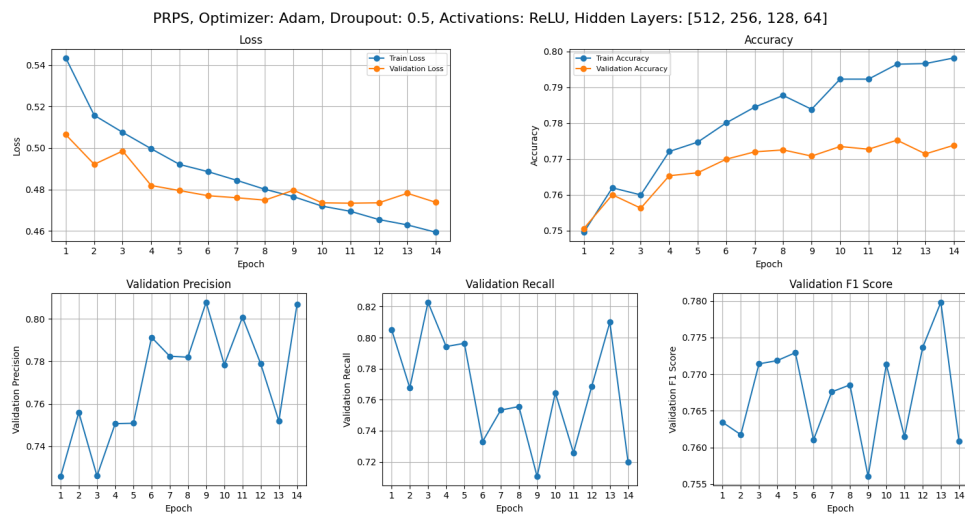


Figure 36: PRPS, 4 hidden layers, [512, 256, 128, 64]

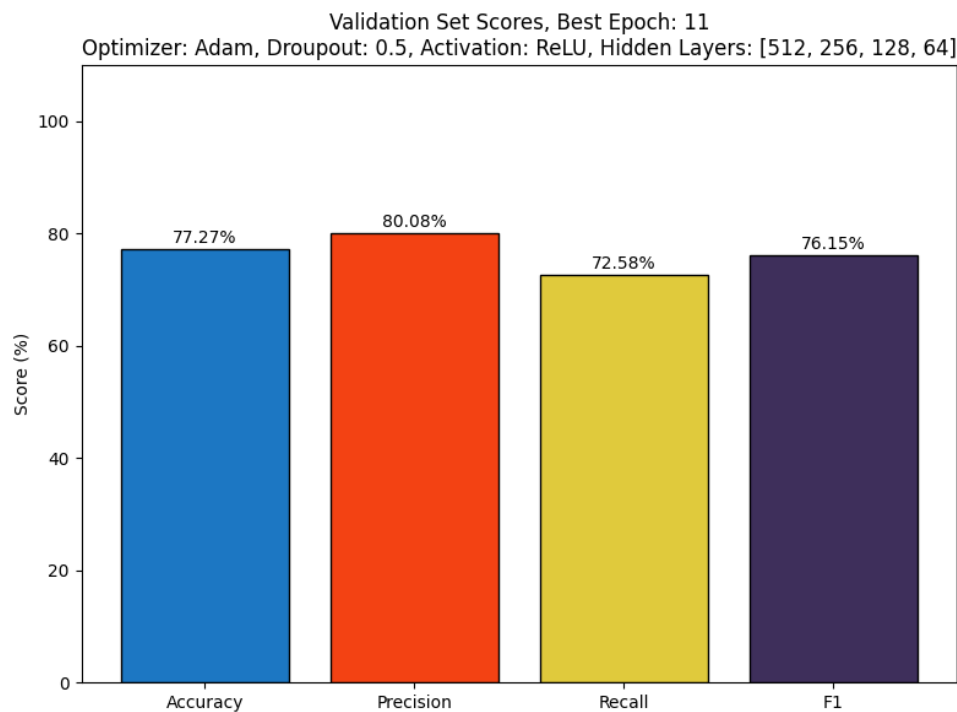


Figure 37: PRPS, best validation scores, 4 hidden layers, [512, 256, 128, 64]

Figures 20-23: The experiments with two hidden layers show that the model's performance improves on both the training set and validation set as the epochs progress. Although both hidden layer size configurations show a stable training across all performance metrics, the [128, 64] configuration has the highest validation set accuracy, with the best model based on minimum validation loss at epoch 22 (early stopping triggered at epoch 25). Early stopping was used, and the validation set accuracy at that point was 77.65%. We observe that the gap between the training accuracy and validation accuracy in the learning curves remains close, with other validation set scores also showing an improving trend as the number of epochs increases. Overall, all the validation set scores (accuracy, precision, recall, f1) remain high close at the final *early-stopping* models.

Figures 24-29: Similar to the experiments with two hidden layers, the models with three hidden layers also show performance improvements on both the training and validation sets as the epochs progress. However, the precision, recall, and F1 scores are less stable than those observed in the two-layer configurations, especially in the [512, 256, 128] architecture. In all three-layer configurations, early stopping was triggered sooner (at epochs 15, 11, and 11 respectively), compared to the two hidden layer configurations. **However, the two-layer configuration achieved a lower validation loss and a better validation accuracy of 77.65%.** We observe that the gap between training and validation accuracy still remains narrow, and other validation metrics continue to show an improving trend as training progresses. Overall, all validation scores (accuracy, precision, recall, F1) remain close at the final *early-stopping* models.

Figures 30-37: Similar to the experiments with three hidden layers and two hidden layers, the four hidden layers follow similar results on both the training and validation losses, which decrease over the epochs. The training accuracy remains close to the validation accuracy throughout training. However, as with the three hidden layer models, the validation set scores for precision, recall, and F1 do not remain as stable as those observed in the two hidden layer experiments. Early stopping is again triggered early (at epochs close to those in the three hidden layer experiments), which is sooner than in the two-layer architecture. **Despite the added depth, the best validation accuracy achieved in this configuration is lower than that of the two hidden layers, while the validation loss remains higher.** Finally, we noticed variability in the precision, recall, and F1 scores at the best epoch of the validation set on the final *early-stopping* models.

Based on the above analysis, we select the two hidden layers configuration [128, 64] as the best performing model, and we continue with further experiments using this architecture, as the deeper architectures did not contribute into a better performing model in our dataset.

4.5. Optimizer Tuning

Optimizers are important hyperparameter methods that are responsible for updating the weights of the neural network during the training process. Once the hidden layers size architecture was selected [128, 64], we experiment with various optimizers shown in Table 4. As we mentioned earlier, the initial model was trained using the Adam optimizer. Figures 38, 39, 40, 41, 42, 43, 44, 45 show the results of the experiments with different optimizer configurations.

Optimizers Tested
NAdam
AdamW
SGD
RMSprop

Table 4: List of tested optimizers

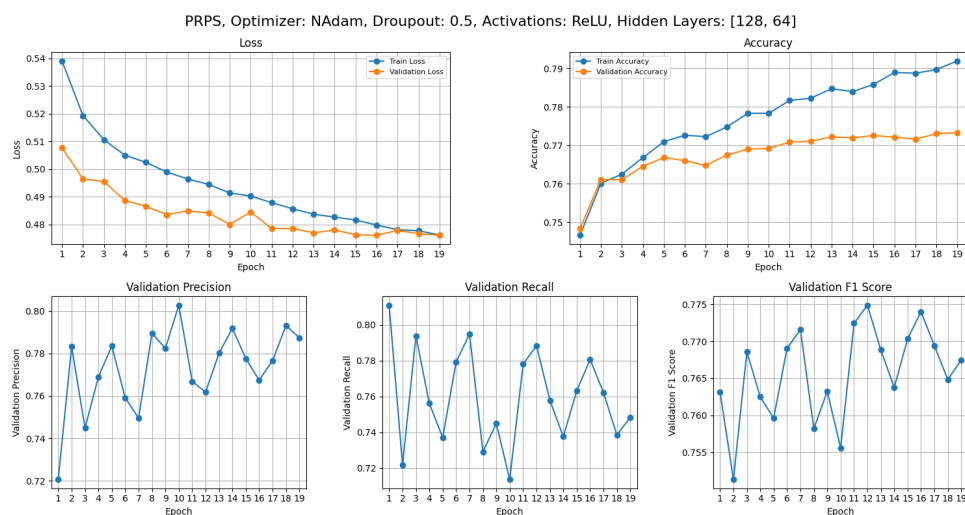


Figure 38: PRPS, 2 hidden layers, [128, 64], NAdam optimizer

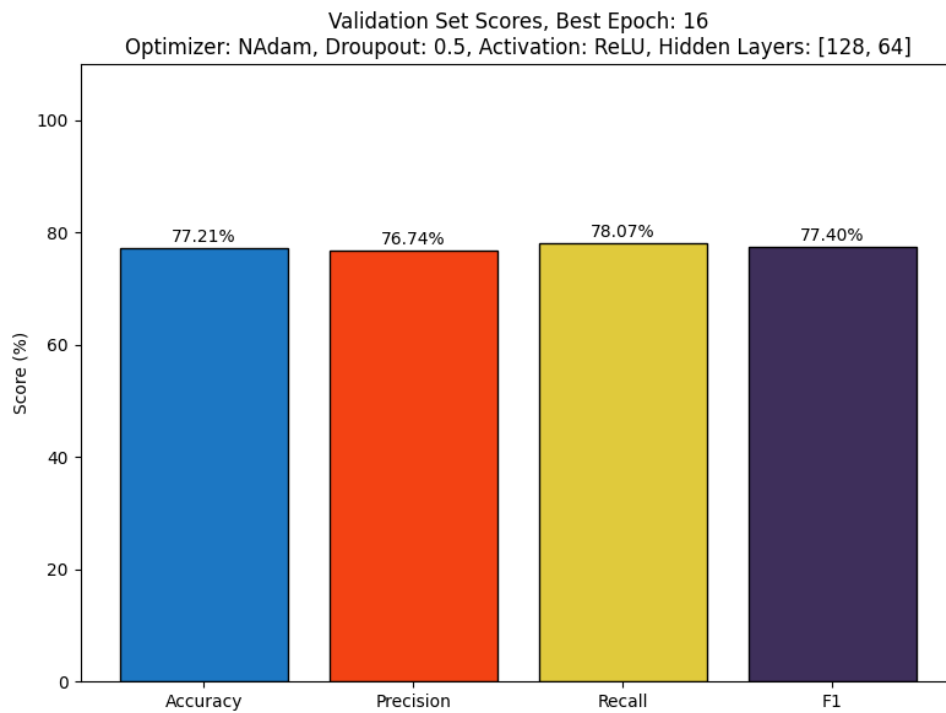


Figure 39: PRPS, best validation scores, 2 hidden layers, [128, 64], NAdam optimizer

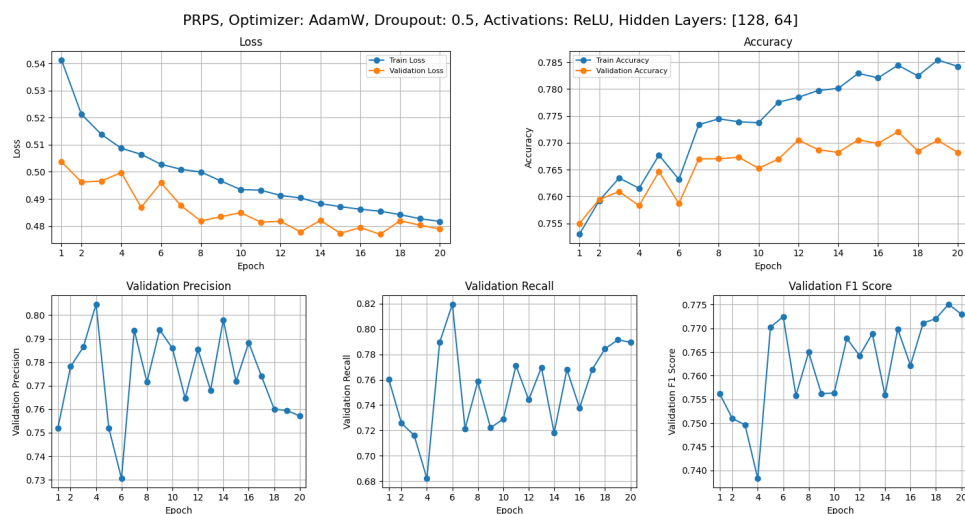


Figure 40: PRPS, 2 hidden layers, [128, 64], AdamW optimizer

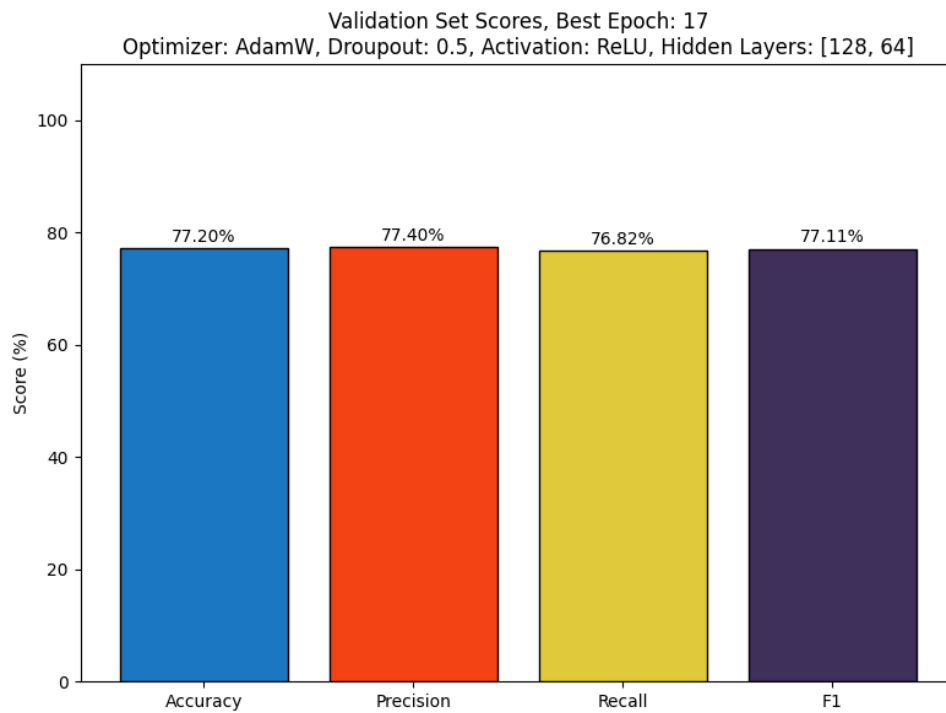


Figure 41: PRPS, best validation scores, 2 hidden layers, [128, 64], AdamW optimizer

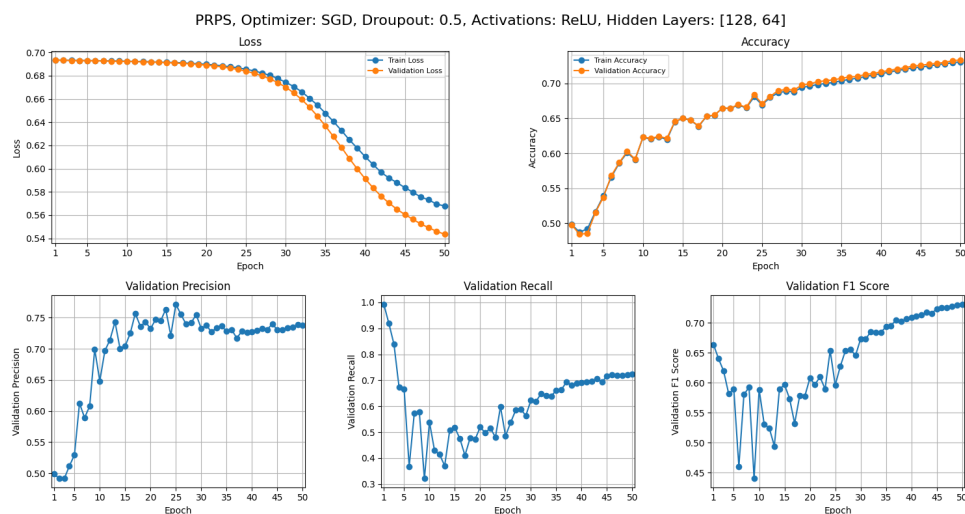


Figure 42: PRPS, 2 hidden layers, [128, 64], SGD optimizer

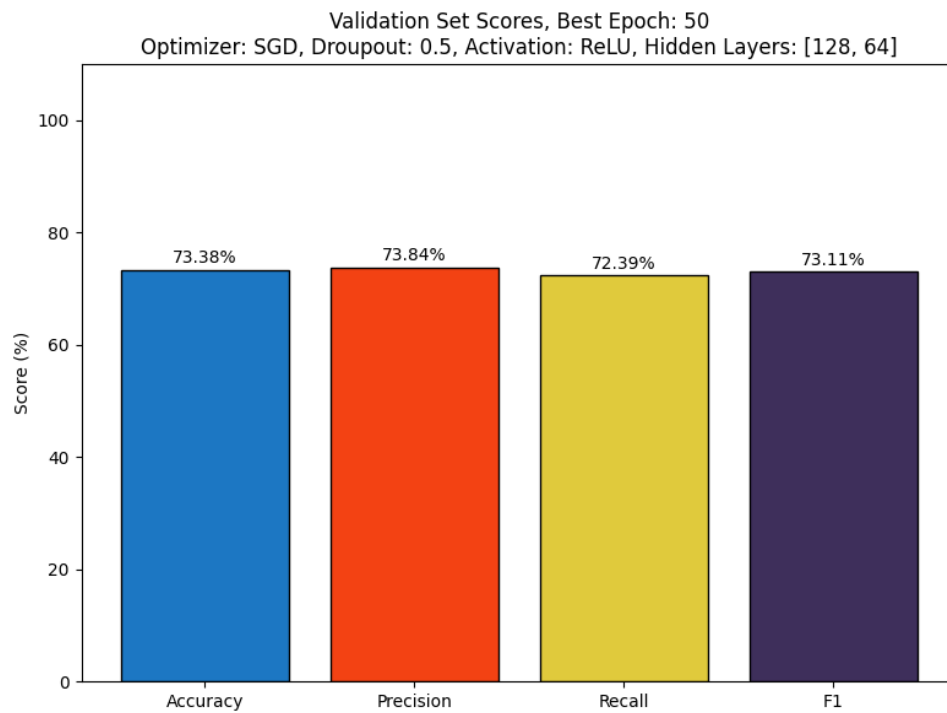


Figure 43: PRPS, best validation scores, 2 hidden layers, [128, 64], SGD optimizer

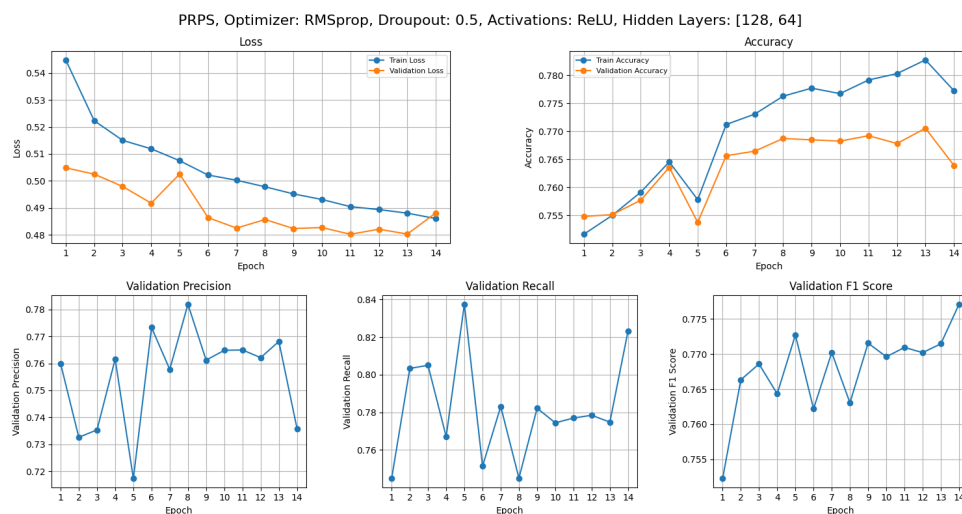


Figure 44: PRPS, 2 hidden layers, [128, 64], RMSprop optimizer

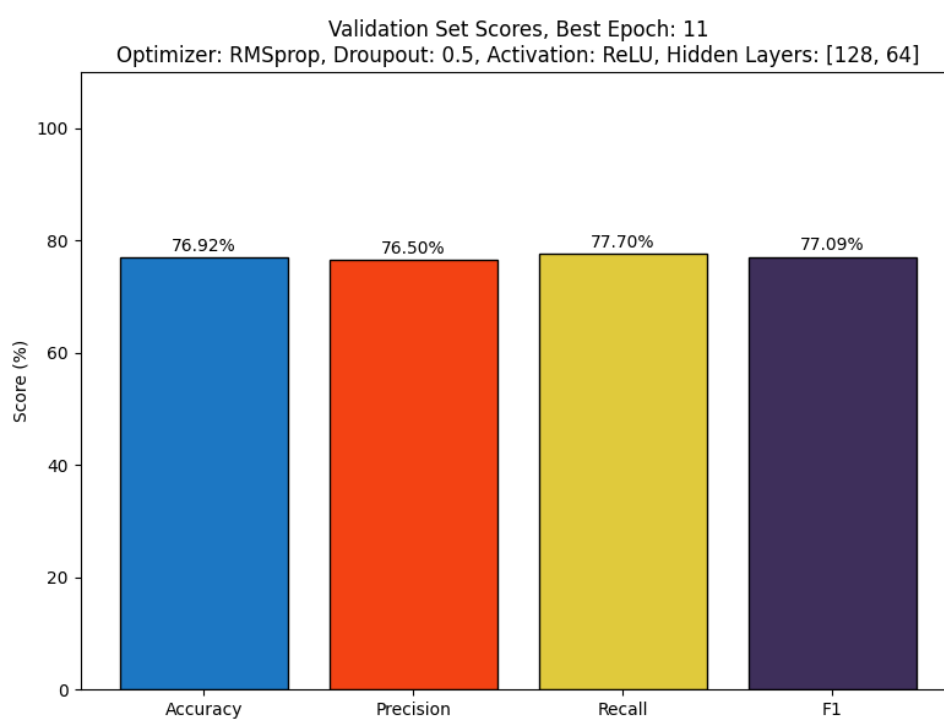


Figure 45: PRPS, best validation scores, 2 hidden layers, [128, 64], RMSprop optimizer

Observing Figures 38 - 45, we notice that **NAdam** is unstable during the training epochs in the precision, recall, and F1 scores of the validation set, leading to a lower validation accuracy score at its best epoch (epoch 16, 77.21%). Furthermore, even though **AdamW** shows a more consistent training process throughout the epochs, stopping at epoch 20 with best epoch being epoch 17, the validation set accuracy score is less than the initial **Adam** optimizer, used in our models so far. Moreover, in the case of the **SGD** optimizer, early stopping is not triggered, and the validation score continues to increase while remaining identical to the training score throughout the epochs. However, after 50 epochs, the final validation set accuracy is significantly lower than **Adam**'s accuracy (77.65%). Finally, **RMSprop**, similar to **AdamW**, shows a stable training process through the epochs. However, after 14 epochs, the best validation set accuracy at epoch 11 is 76.92%, which is again significantly lower than the initial **Adam** optimizer.

4.6. Batch Normalization

To further stabilize training and improve convergence, batch normalization layers [11] were introduced after each hidden layer. Figures 46 and 47 show the learning curves of the training and validation sets, as well as the validation set scores at the best epoch (based on validation loss). We notice that early stopping is triggered after 13 epochs, with the best model saved at epoch 10. The validation accuracy remains identical to the model without batch normalization, and the other validation scores are nearly the same, despite the total number of training epochs being reduced from 25 to just 13. We observe stable training throughout the epochs until epoch 10, which is the best epoch of the model.

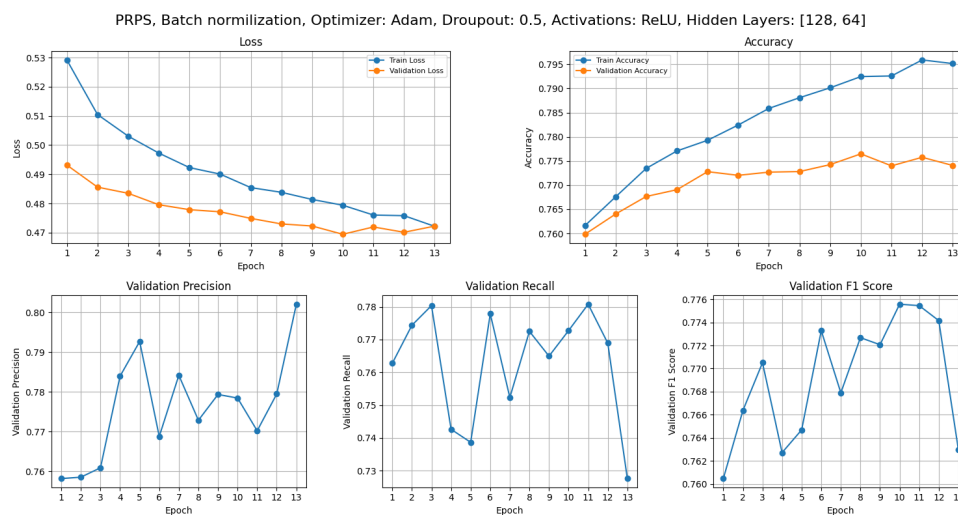


Figure 46: PRPS, 2 hidden layers, [128, 64], Adam optimizer, batch normalization

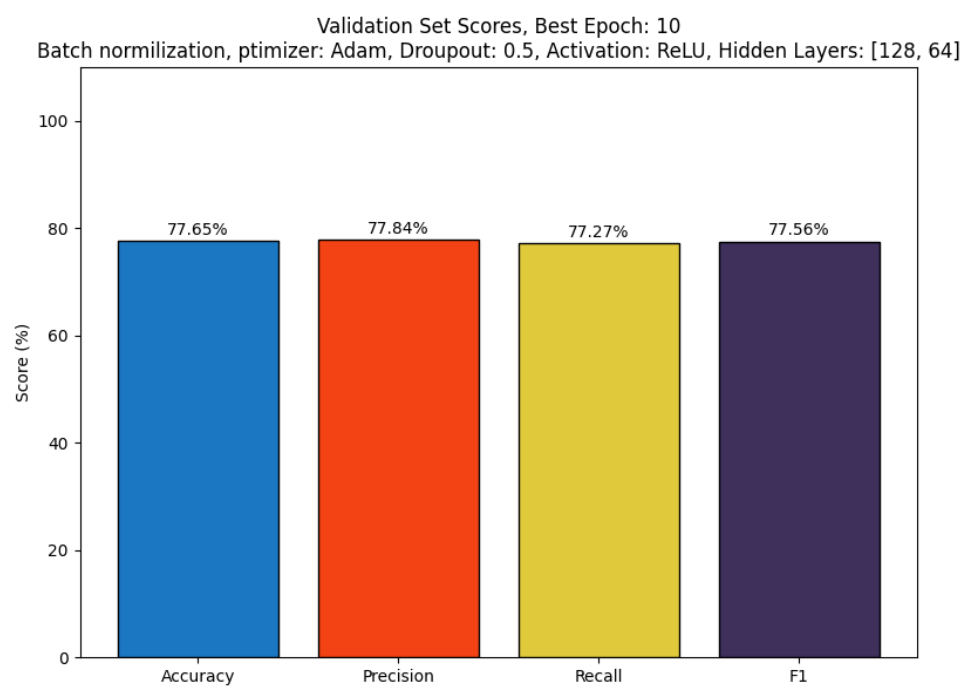


Figure 47: PRPS, best validation scores, 2 hidden layers, [128, 64], Adam optimizer, batch normalization

5. Optuna and Final Results

5.1. Running an Optuna study

Finally, the best-performing model architecture and training configuration (based on preprocessing, dropout, optimizer, and normalization) was used as a baseline for hyperparameter optimization with Optuna. Table 5 shows the best model architecture and hyperparameter configuration before using Optuna optimization framework.

Parameter	Configuration
Hidden Layers	2 hidden layers [128, 64]
Activation Function	ReLU
Optimizer	Adam
Batch Normalization	Yes
Early Stopping	Triggered at epoch 13
Best Epoch	Epoch 10
Validation Accuracy	77.65%

Table 5: Best model configuration before using Optuna.

Optuna was configured to run 40 trials, starting from our best model so far (Table 5). It explored combinations of the following hyperparameters:

- Dropout rates ranging from 0.2 to 0.5
- Learning rates from 1×10^{-5} to 1×10^{-2} (log scale)
- Activation functions [nn.ReLU, nn.LeakyReLU, nn.ELU, nn.Sigmoid, nn.SiLU] applied to all hidden layers

The goal of the Optuna study was to minimize the validation loss and find a better-generalized model with improved validation accuracy. Similar to the previous experiments, early stopping was enabled to store the model at the epoch with the lowest validation loss.

The final model configuration after Optuna hyperparameter optimization is presented in Table 6. This configuration includes 2 hidden layers with sizes [128, 64], SiLU activation for the first layer, and ELU activation for the second layer. The model uses a dropout rate of 0.384, a learning rate of 2.24×10^{-4} , and the Adam optimizer with batch normalization. Early stopping was triggered at epoch 31, with the best epoch being epoch 28, having a final **validation accuracy of 77.88%**. Figures 48 and 49 show the learning curve and the performance scores, for this optimized model. We notice that validation set score is close to the training set score with validation loss decreasing through the epochs. Therefore, the model does not have any overfit or underfit issues. Finally all the validation set scores (accuracy, precision, recall and F1) remain stable and close between them.

Finally, Figures 50 and 51 show the ROC curve [15] and confusion matrix[14] of the Optuna model using the validation dataset. The AUC of 0.8 shows that the model has a solid classification performance and it is able to distinguish between positive and negative tweets. The confusion matrix shows us the distribution of misclassification presenting the false negative and false positive predictions of our final model. We

Parameter	Value
Hidden Layers	2 hidden layers [128, 64]
Activation Function (Layer 1)	SiLU
Activation Function (Layer 2)	ELU
Dropout Rate	0.384
Learning Rate	2.24×10^{-4}
Optimizer	Adam
Batch Normalization	Yes
Batch Size	128
Early Stopping	Triggered at epoch 31
Best Epoch	Epoch 28
Validation Accuracy	77.88%

Table 6: Final Model Configuration after Optuna Hyperparameter Optimization

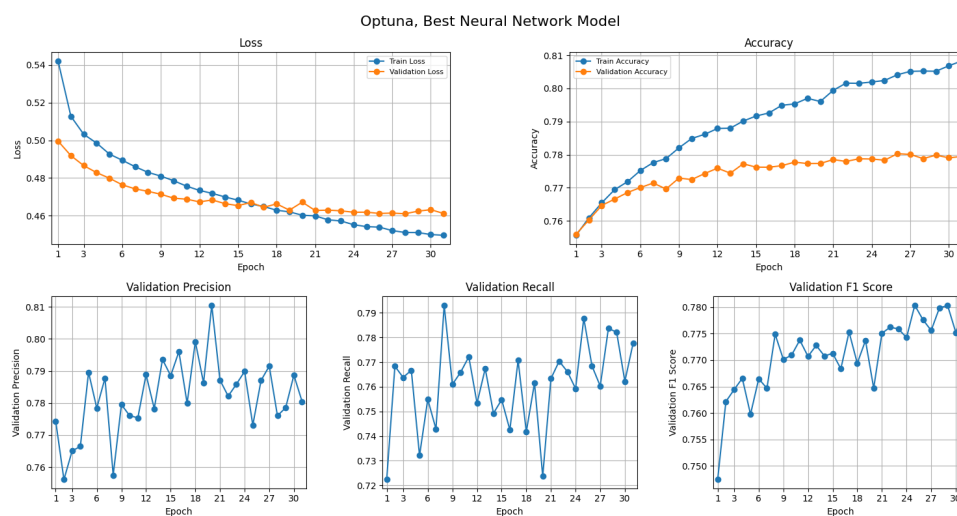


Figure 48: PRPS, 2 hidden layers, [128, 64], Adam optimizer, batch normilization, activation functions for the hidden layers are SiLU for the first layer, ELU for the second layer, with a dropout rate of 0.384 and a learning rate of 0.00022.

notice that the number of false positives and false negatives are almost identical in our validation set experiment.

The test set score (Kaggle submission) is 77.654%, which is very close to the validation set score. Therefore, our final model has generalized well. Multiple submissions were made to Kaggle, with the highest test set score reaching 77.961%, though the selected model is based on our validation set score and not the competition score (as instructed).

5.2. Comparison with the first project & future work

Compared to the first project, using a feed-forward neural network with Word2Vec embeddings required significantly more work to optimize the hyperparameters and the model's architecture to achieve scores close to those of the logistic regression model, which still achieves better validation set accuracy at 80.37%. Furthermore, the results may be slightly worse due to the feature extraction mechanism, where after transform-

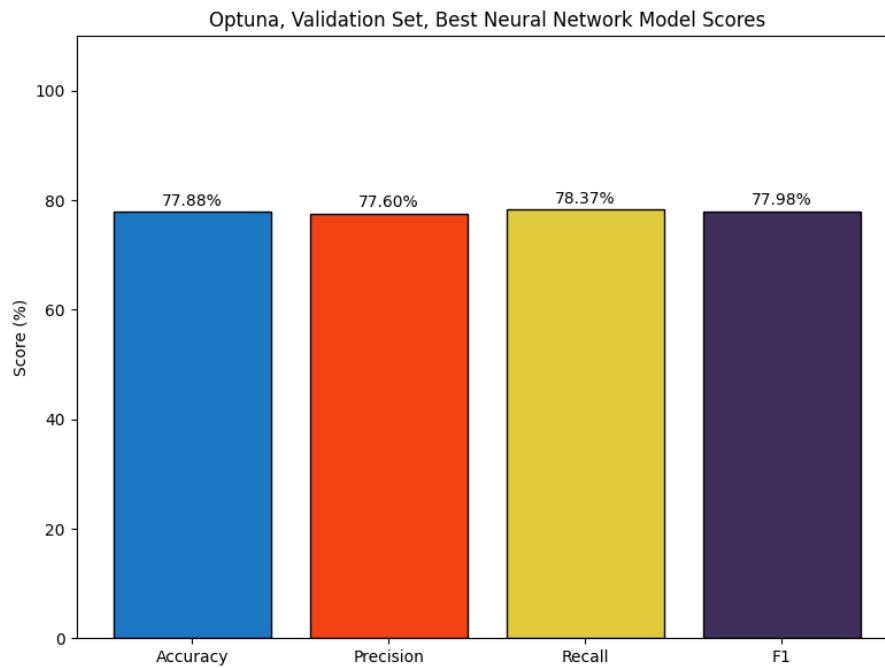


Figure 49: PRPS, validation set scores, 2 hidden layers, [128, 64], Adam optimizer, batch normalization, activation functions for the hidden layers are SiLU for the first layer, ELU for the second layer, with a dropout rate of 0.384 and a learning rate of 0.00022.

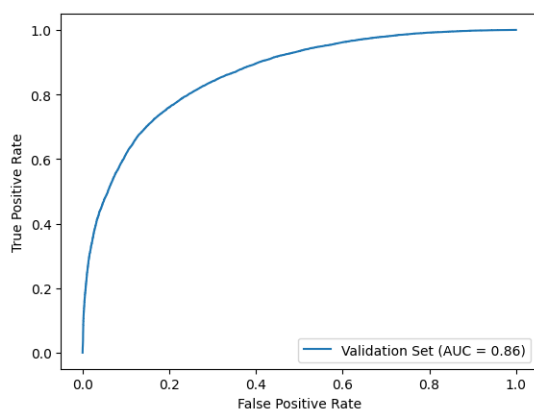


Figure 50: PRPS, ROC curve using the optimized Optuna model.

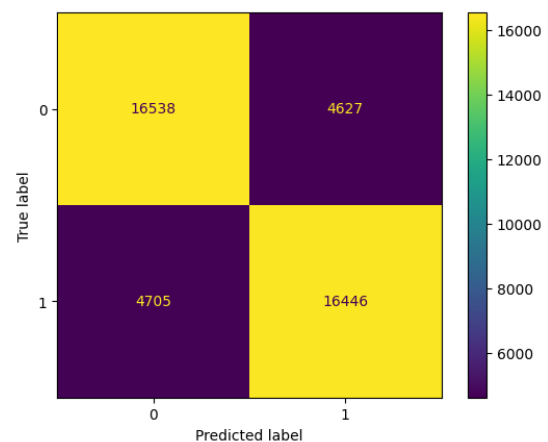


Figure 51: PRPS, confusion matrix, using the optimized Optuna model.

ing tokens into vector embeddings, we calculate the mean value. This may lead to the loss of frequency information, which is maintained and used in the TF-IDF methodology, as it gives more *attention* to frequent tokens. In addition, the neural network required more time to train, and the number of parameters to evaluate was higher than that of the logistic regression model. As future work, we would like to run more Optuna trials with a larger hyperparameter pool and also test different batch sizes. In this project, we first searched through a range of parameters to find a well-optimized base model, and then ran Optuna trials with a smaller hyperparameter pool to find a better feed forward model within a reasonable time (due to GPU limitations). **Note**

that although seeds were set to ALL the python packages based on the corresponding documentation (including CUDA), still slightly different results to the neural networks on Kaggle may be seen on each execution, therefore in each execution, the best model is chosen in an automated way step-by-step.

6. Bibliography

References

- [1] amueller. Word cloud by andreas mueller. https://amueller.github.io/word_cloud/.
- [2] geeksforgeeks. Check if email address is valid or not in python. <https://www.geeksforgeeks.org/check-if-email-address-valid-or-not-in-python/>.
- [3] geeksforgeeks. geeksforgeeks early-stopping. <https://www.geeksforgeeks.org/how-to-handle-overfitting-in-pytorch-models-using-early-stopping/>.
- [4] geeksforgeeks. Python nltk: Tweettokenizer. <https://www.geeksforgeeks.org/python-nltk-nltk-tweettokenizer/>.
- [5] geeksforgeeks. Python regex lookbehind. <https://www.geeksforgeeks.org/python-regex-lookbehind/>.
- [6] geeksforgeeks. Text preprocessing for nlp tasks. <https://www.geeksforgeeks.org/text-preprocessing-for-nlp-tasks/>.
- [7] google. Google: Googlenews embeddings. <https://code.google.com/archive/p/word2vec/>.
- [8] kaggle. kaggle slang words for nlp. <https://www.kaggle.com/code/nmaguette/up-to-date-list-of-slans-for-text-preprocessing>.
- [9] matplotlib. matplotlib examples. <https://matplotlib.org/stable/gallery/index.html>.
- [10] optuna. Optuna pythonic search space. https://optuna.readthedocs.io/en/stable/tutorial/10_key_features/002_configurations.html.
- [11] pytorch. pytorch batch-normalization. <https://pytorch.org/docs/stable/generated/torch.nn.BatchNorm1d.html>.
- [12] pytorch. pytorch bcewithlogitsloss. <https://pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html>.
- [13] pytorch. Pytorch dropout. <https://pytorch.org/docs/stable/generated/torch.nn.Dropout.html>.
- [14] sklearn. sklearn metrics confusion matrix. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html.
- [15] sklearn. sklearn metrics roc curve. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_curve.html.

- [16] stanford. stanford glove: Global vectors for word representation. <https://nlp.stanford.edu/projects/glove/>.